

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output. The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset **Source:** UCI Machine Learning Repository **Download Link:** <https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance

- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization

- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Source Code

<https://colab.research.google.com/drive/1hAPpA0Xz17SAVMKtX80QwS7k1cb2vW0E?usp=sharing>

11. Experimental Results

Task 1: Dataset Exploration

- Dataset Size: 1372×5
- Number of Features: 4
- Number of Classes: 2
- Missing Values: 0
- Dataset was complete and required no data cleaning.

Task 2: Exploratory Data Analysis (EDA)

- Feature distributions are non-uniform.

- Variance is the most discriminative feature.
- Scatter plot shows good separation between classes.
- Few outliers are present but do not significantly affect classification.

Task 3: Data Preprocessing

- Training Samples: 1097
- Testing Samples: 275
- Training Feature Shape: (1097, 4)
- Testing Feature Shape: (275, 4)
- All features were successfully normalized before training.

Task 4: Perceptron Implementation

- Learning Rate: 0.01
- Epochs: 100
- Initial Misclassified Samples: 47
- Final Misclassified Samples: 12
- Final Bias: -0.13

Task 5: Model Training

- Training error reduced from 47 to 12 misclassified samples.
- Model parameters converged after several epochs.
- Weight values stabilized during training.
- Bias converged to approximately -0.13 .
- Decision boundary successfully separated most training samples.
- Different learning rates produced similar classification performance.

Task 6: Model Evaluation

Metric	Value
Accuracy	98.18%
Precision	96.06%
Recall	100.00%
F1-score	97.99%

Confusion Matrix

Actual / Predicted	Class 0	Class 1
Class 0	148	5
Class 1	0	122

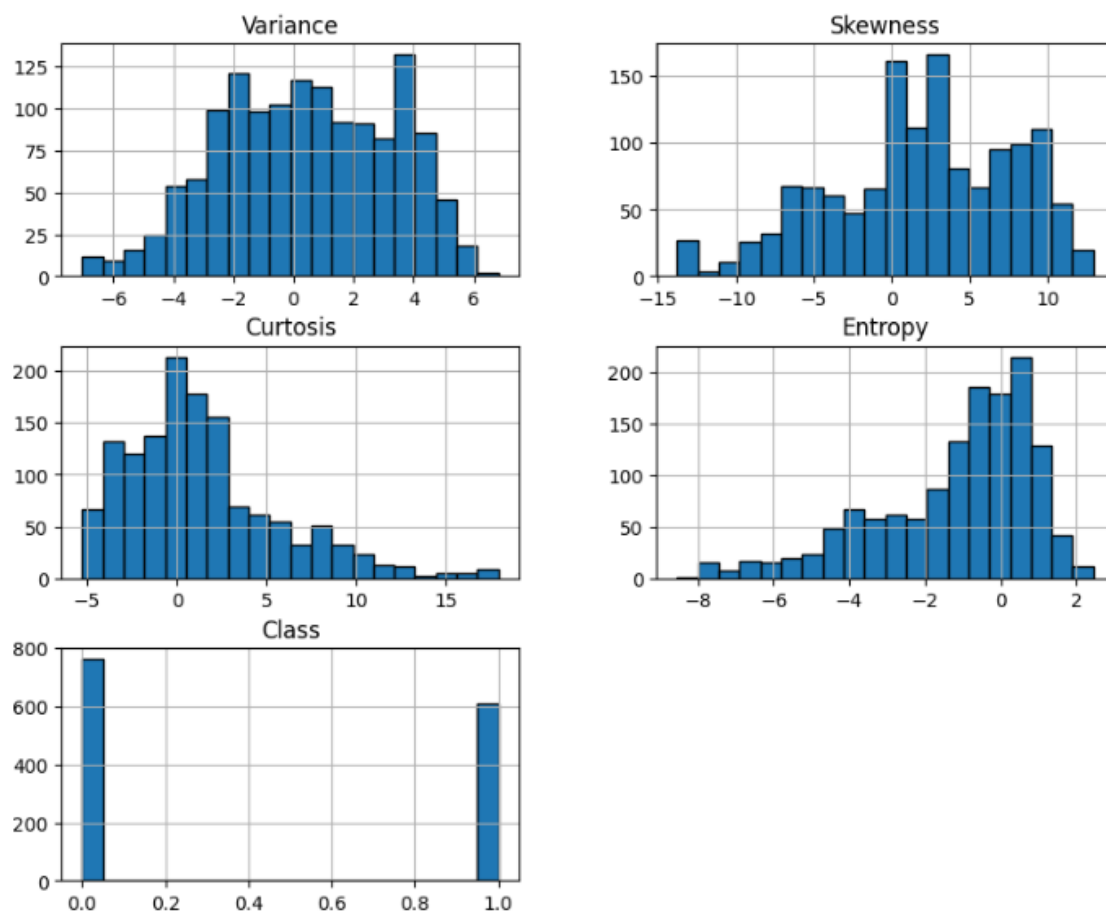
Classification Report

Class	Precision	Recall	F1-score	Support
Class 0	1.00	0.97	0.98	153
Class 1	0.96	1.00	0.98	122
Accuracy	98.18% (275 Samples)			
Macro Avg	0.98	0.98	0.98	275
Weighted Avg	0.98	0.98	0.98	275

1 Generated Plots with Interpretation

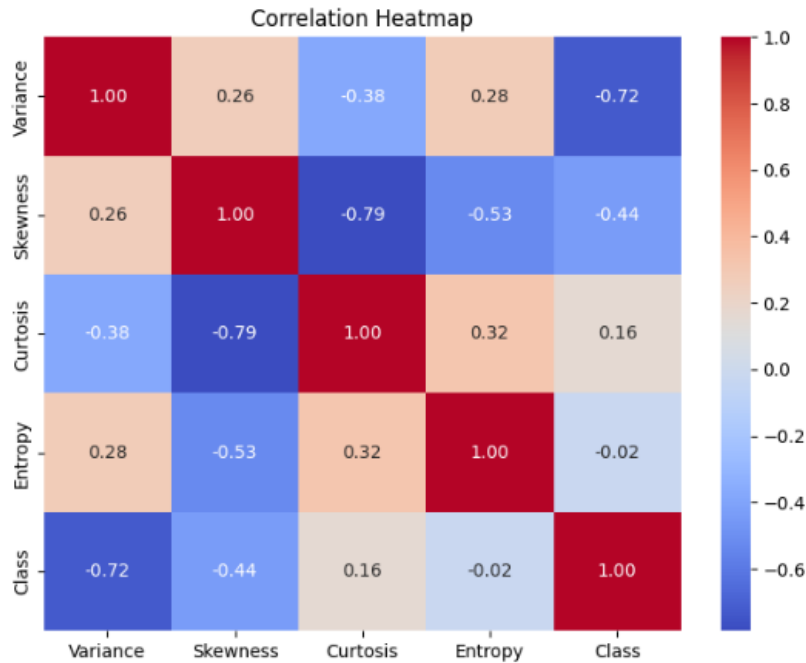
1. Feature Histograms

Feature Histograms



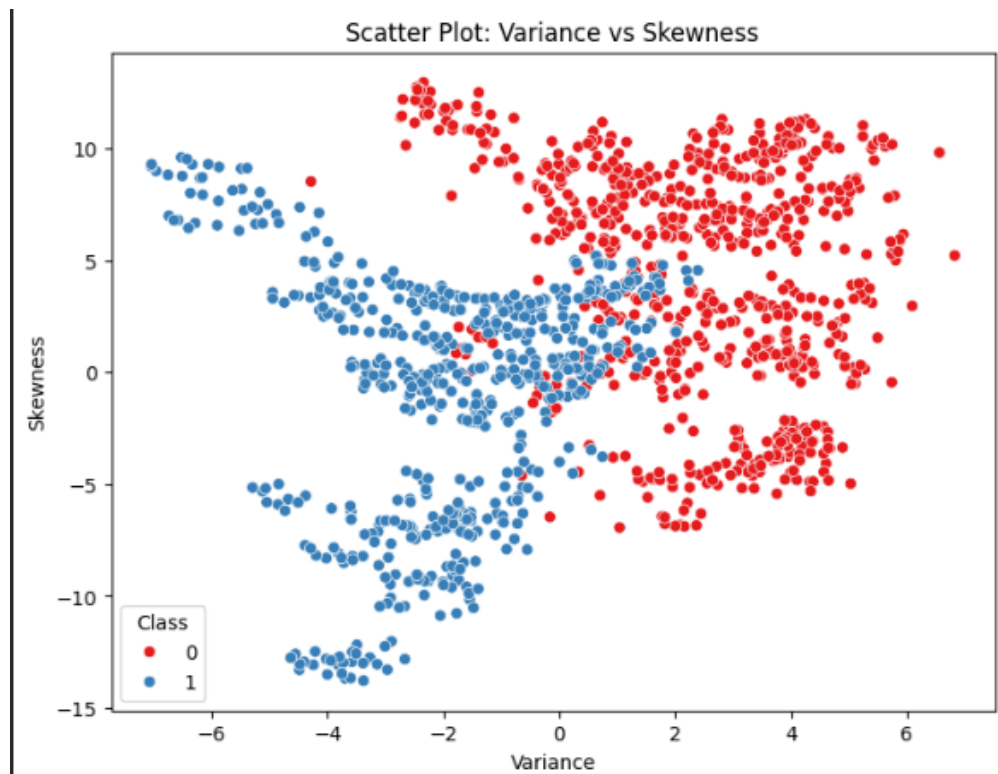
The histograms indicate that the four features have distinct distributions and are not normally distributed. kurtosis shows that a rightskewed distribution with a very few highvalue observations, while Variance, skewness, and entropy are more distributed across their respective ranges. This diversity in feature distributions provides meaningful information for differntaiting between the two classes.

2. Correlation Heatmap



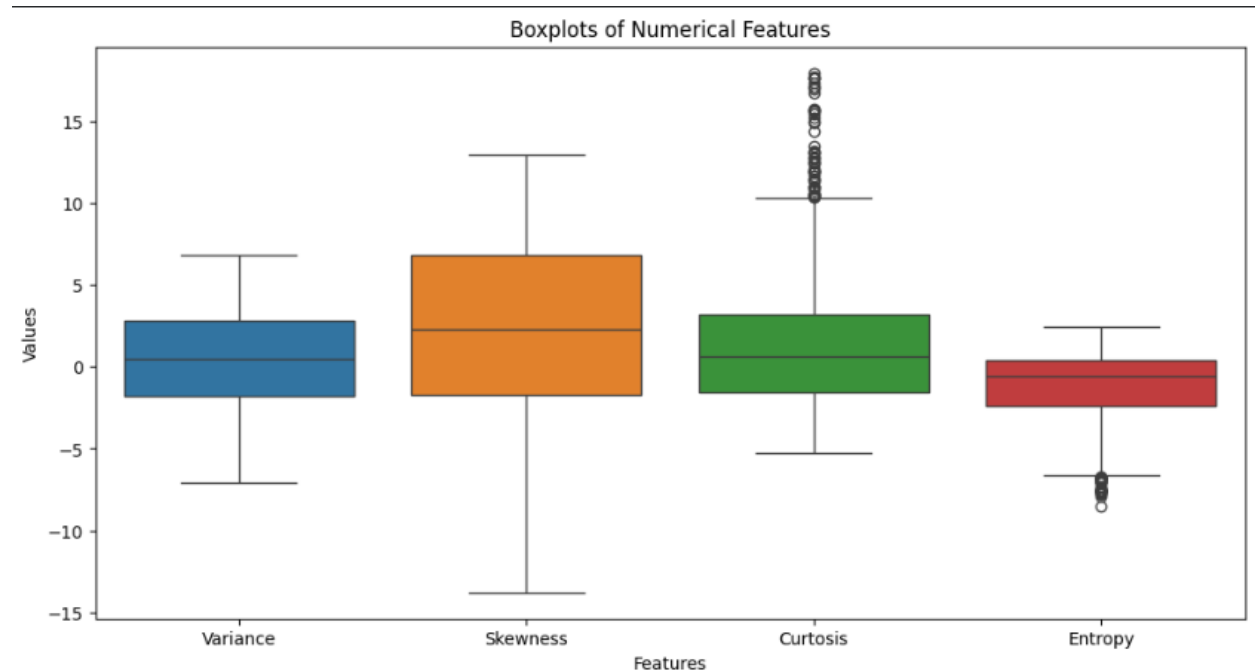
The heatmap shows that variance has the strongest correlation with the target class (-0.72), making it the most significant feature for classification. Skewness is moderately correlated with the class, while kurtosis and entropy have relatively weaker relationships. A strong negative correlation is also observed between skewness and kurtosis.

3. Scatter Plot



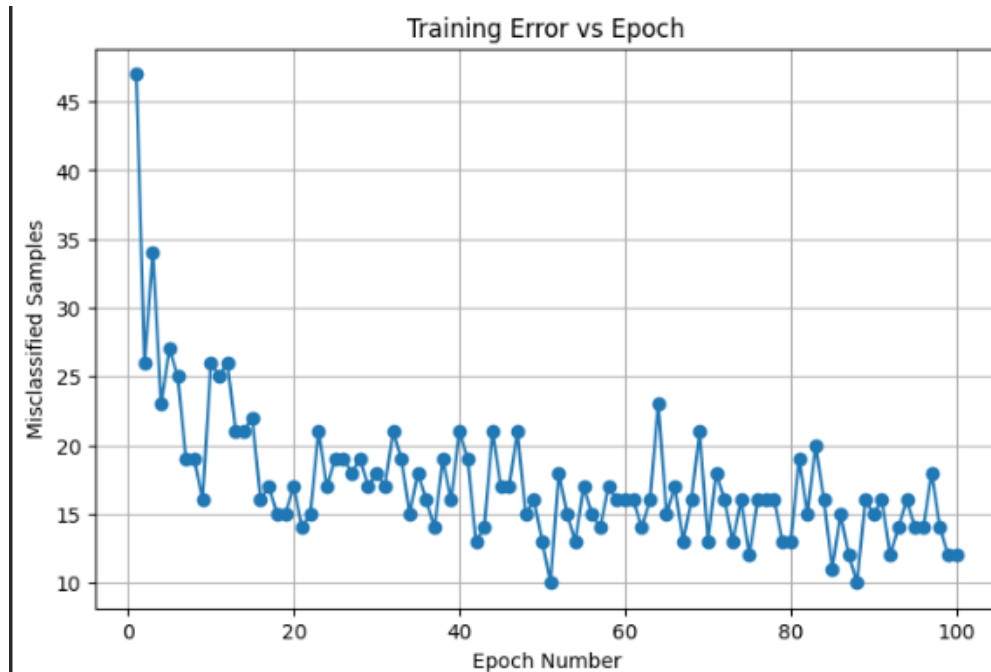
The scatter plot shows that the two classes are very largely separated based on the variance and the skewness features. while a few samples from the both classes overlap around the boundary region, and the majority of the observations form welldefined clusters. This suggests that the dataset is nearly a linearly separable, making it suitable for the classification using a single layer Perceptron.

4. Boxplots



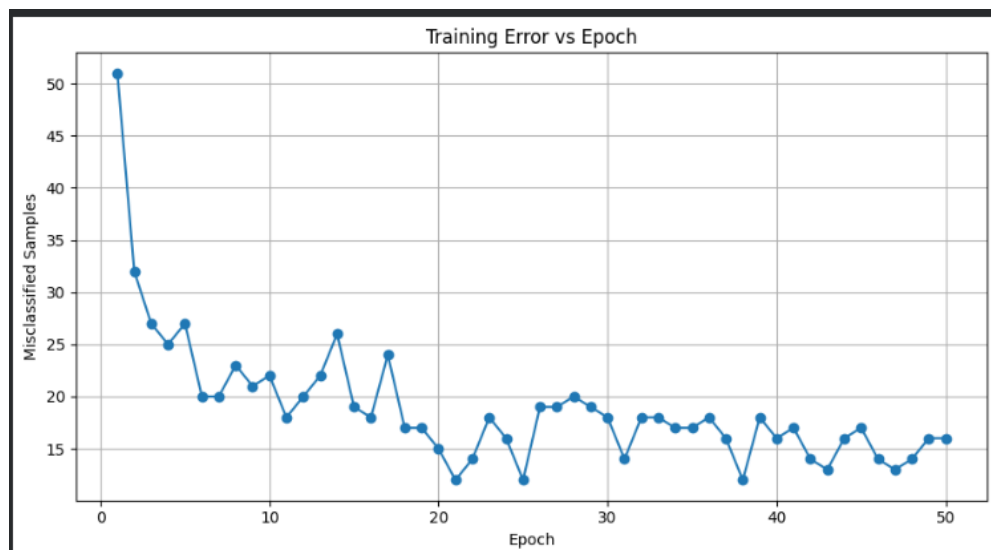
The boxplots provide a summary of the distribution of each feature through their medians, interquartile range, and outliers. skewness has the largest spread, whereas the kurtosis contains a numerous high value outliers. Entropy shows a few low value outliers, while variance displays comparatively lesser extreme observations. since these outliers belong to the original dataset, they were preserved during preprocessing.

5. Training Error vs Epoch



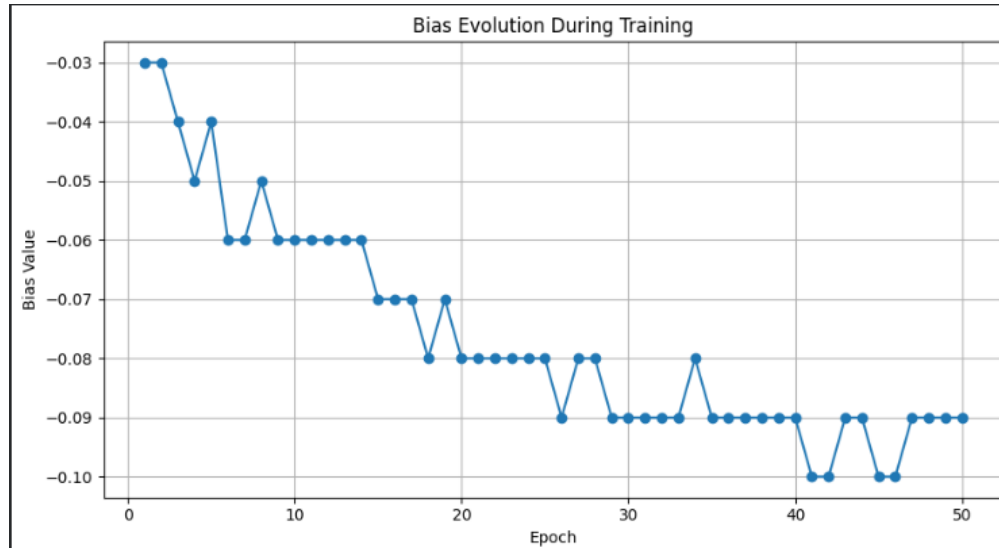
The training error decreases during the initial epochs and gradually stabilizes as the training progresses. minor fluctuations are observed in the later epochs, but the overall trend indicates that the Perceptron has converged and they learned an effective decision boundary.

6. Weight Evolution



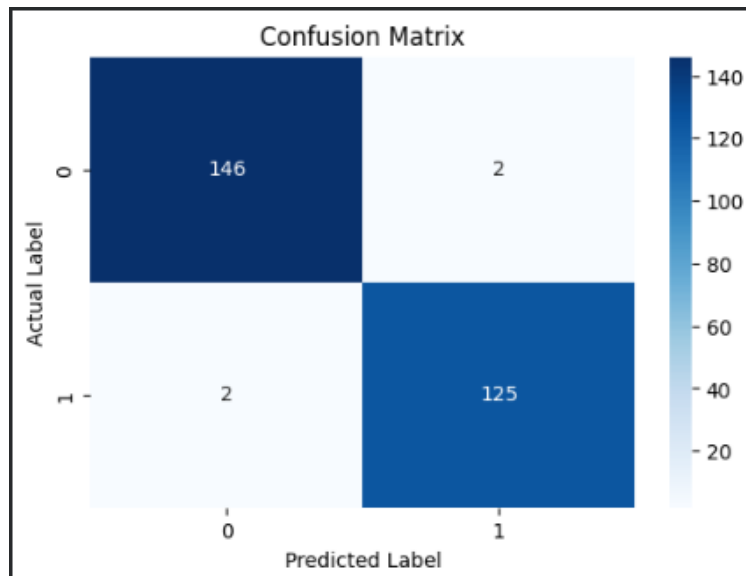
The weight values change noticeably during the early stages of training because the model is still learning the patterns present in the dataset. As more epochs are completed, the updates become less significant and the weights begin to stabilize. This indicates that the Perceptron has adapted to the training data and found a suitable set of feature weights for making predictions.

7. Bias Evolution



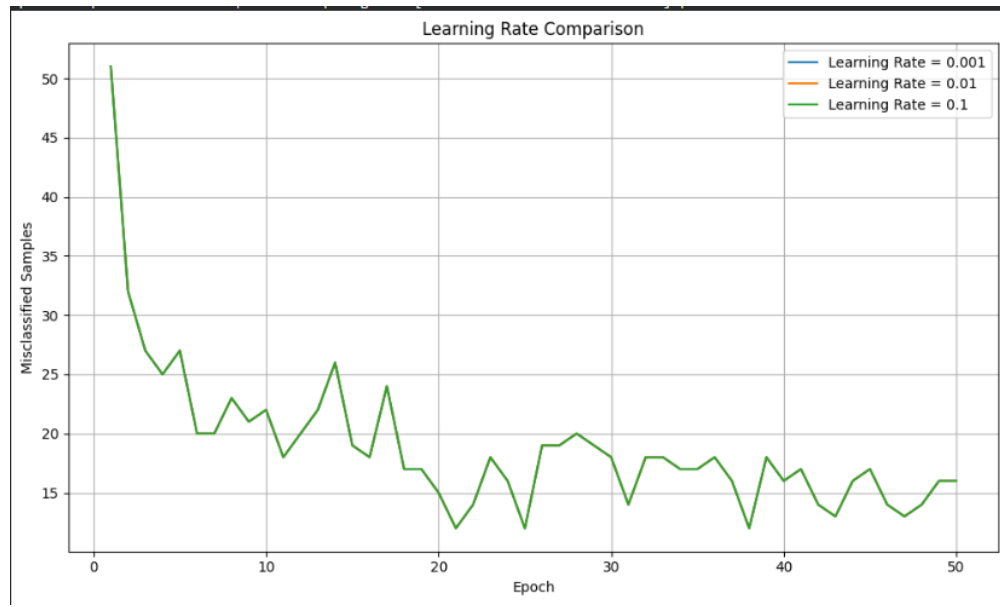
initially, the bias changes noticeably because the model is adapting to the training samples. after the several epochs, the bias settles with only a few minor changes, suggesting that the learning process has converged. This stable behaviour reflects that the model is making consistent predictions.

8. Confusion Matrix



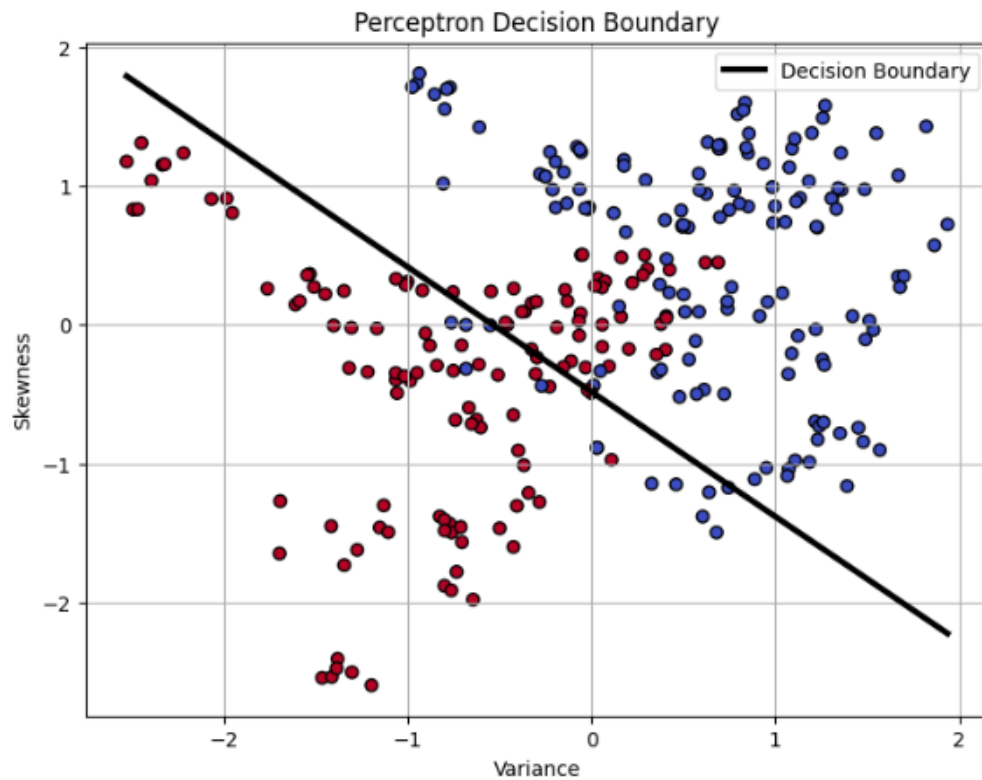
The confusion matrix shows that the majority of the test samples were classified correctly with only a few incorrect predictions. This resulted in an overall accuracy of **98.18%**, indicating that the single layer Perceptron performs effectively on the banknote authentication dataset.

9. Learning Rate Comparison



The learning rate comparison demonstrates that the Perceptron converges successfully for all the selected learning rates. Even though the learning process varies slightly during training, the final classification performance remains the same. This shows that the model is not highly sensitive to the choice of learning rate for this dataset.

10. Decision Boundary



The graph shows that all three learning rates gradually reduce the training error and eventually reach a stable point. While the learning curves differ slightly during the initial epochs, the final performance remains almost identical. This suggests that the Perceptron is able to learn the patterns in the dataset effectively regardless of the chosen learning rate.

8. Performance Tables

Training Summary

Parameter	Value
Dataset Size	1372 Samples (1372×5)
Train/Test Split	80% Training (1097 samples), 20% Testing (275 samples)
Learning Rate	0.01
Epochs	50
Final Weights	[-0.2040, -0.2218, -0.1995, -0.0226]
Final Bias	-0.09
Accuracy	98.18%
Precision	96.06%
Recall	100.00%
F1-score	97.99%

9. Additional Tasks

1. Comparison of Step and Sigmoid Activation Functions

The step activation function produces only the two possible outputs, either 0 or 1, making it suitable for basic binary classification. However, since it is not differentiable, it cannot be used for gradient-based learning algorithms such as the backpropagation. The Sigmoid activation function generates continuous values between 0 and 1 and is differentiable, allowing the network to update its parameters efficiently during training. Although modern deep learning models often use activation functions like ReLU, sigmoid is still widely used for binary output layers.

2. Comparison with Scikit-learn's Perceptron

The perceptron implemented from scratch achieved an accuracy of **98.18%** on the Banknote Authentication dataset, while the Scikit-learn implementation achieved an accuracy of **96.73%**. Although Scikit-learn provides a faster and optimized implementation with built-in functionalities, implementing the algorithm manually offers a deeper understanding of the weight update mechanism, bias adjustment, and the overall learning process of the Perceptron.

3. Learning Rate Comparison

The model was trained using three different learning rates: **0.001**, **0.01**, and **0.1**. All three learning rates were able to learn the dataset successfully and produced similar classification performance. The main difference was observed in the learning behaviour during training, while the final results remained nearly identical. This shows that the Perceptron performs consistently on this dataset for a range of learning rates.

4. Why a Single Layer Perceptron Cannot Solve the XOR Problem

A Single Layer Perceptron can learn only linear decision boundaries. In the XOR problem, the two classes cannot be separated using a single straight line because the data points are arranged in a non-linear pattern. As a result, the Perceptron fails to classify all XOR inputs correctly. To solve this problem, a Multi-Layer Perceptron with one or more hidden layers is required to learn the non-linear relationship between the inputs.

5. Effect of Feature Normalization on Model Convergence

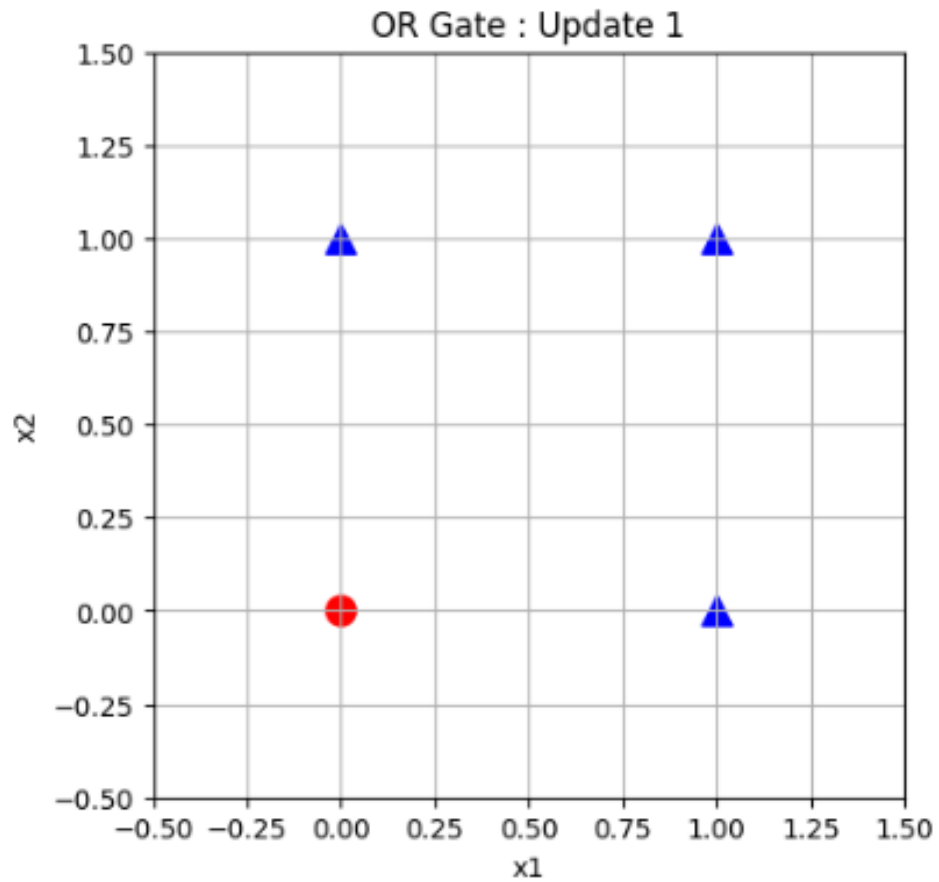
Feature normalization transforms the input features onto a similar scale before training. This prevents features with larger numerical values from having a greater influence on the weight updates. In this experiment, normalization helped the Perceptron learn more steadily and contributed to stable convergence during training. Using normalized features, the model achieved an accuracy of **98.18%**, demonstrating the effectiveness of preprocessing for improving the learning process.

1.1 Implementation of Perceptron Learning Algorithm

Question:

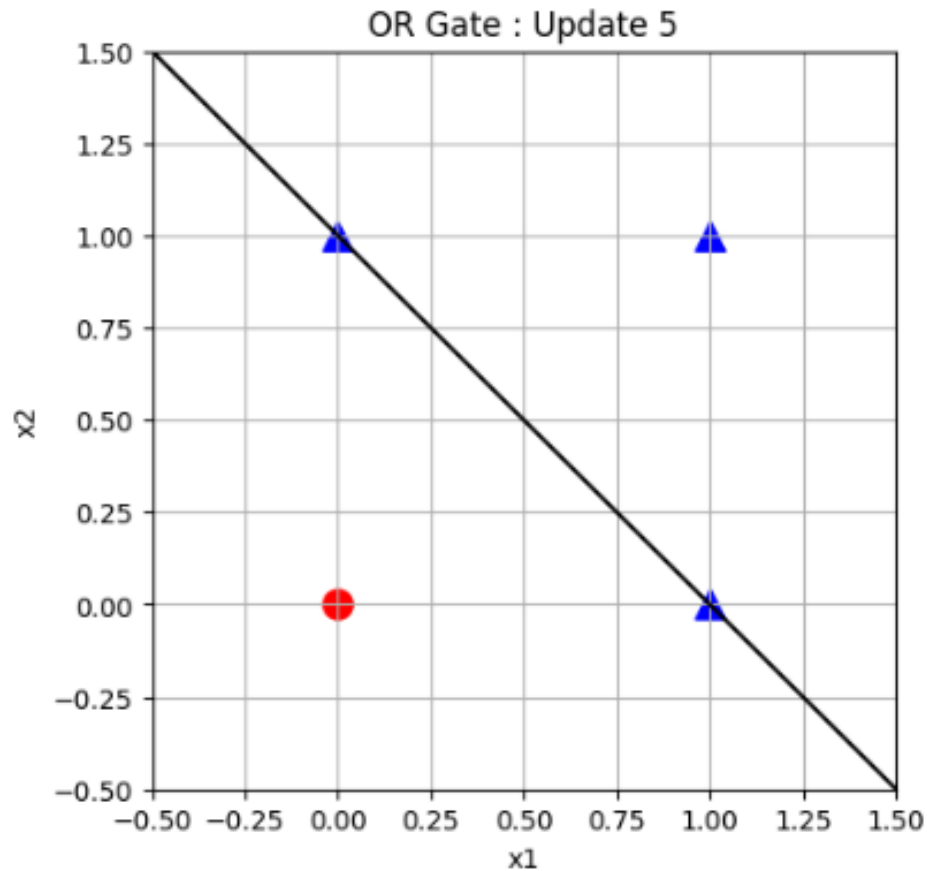
Code of the perceptron learning algorithm for OR, NOT, AND, and XOR gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using the Python built-in packages. analyze the learning behavior for each logic gate. (K4)

1. OR Gate – Update 1



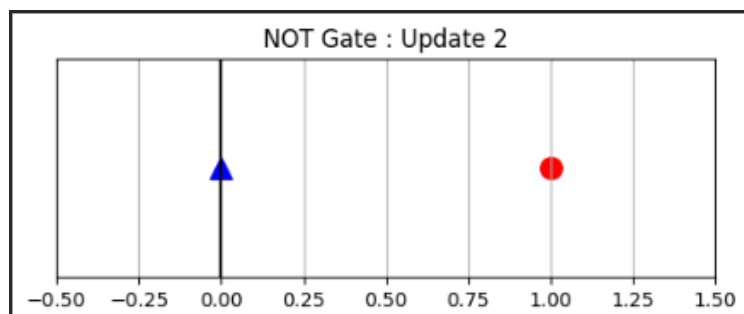
At the beginning of training, the perceptron has not yet learned the correct pattern of the OR gate. The decision boundary does not separate the inputs properly because the weights have only been updated once. This is expected during the initial stage, as the model is still learning from the training examples.

2. OR Gate – Final Update



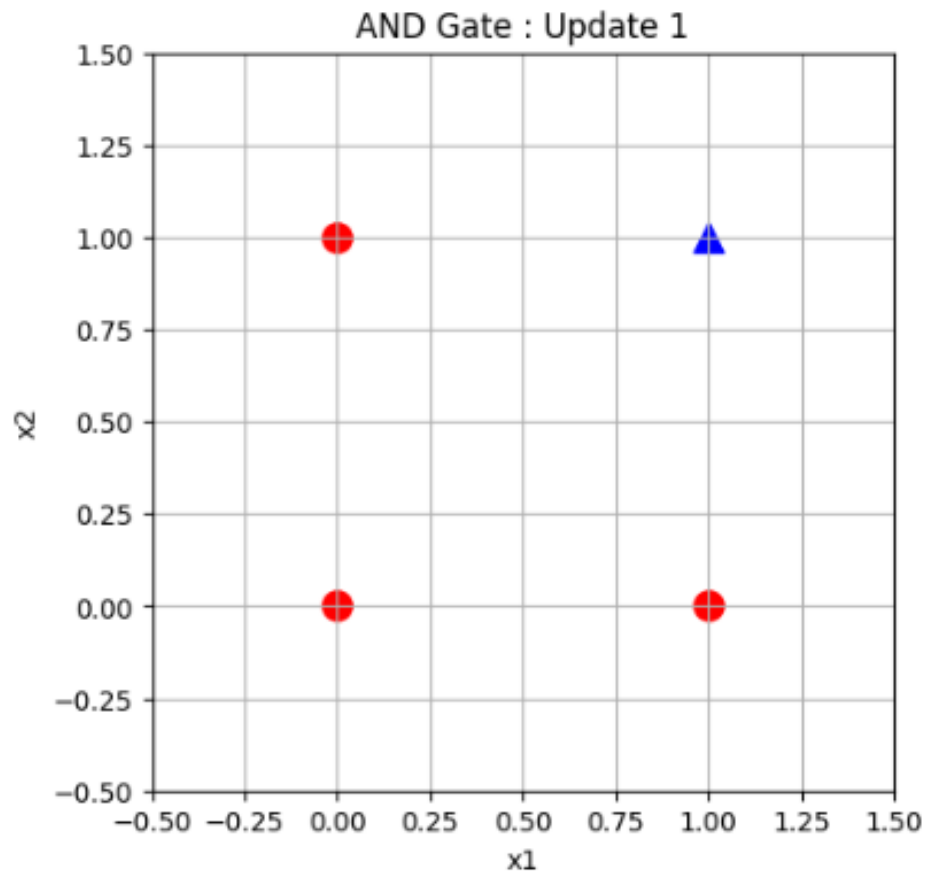
After a few updates, the decision boundary moves to the correct position and all four input combinations are classified accurately. The perceptron finishes training with weights **[1, 1]** and a bias of **-1**. This shows that the OR gate is linearly separable and can be learned successfully using a Single Layer Perceptron.

3. NOT Gate – Final Decision Boundary



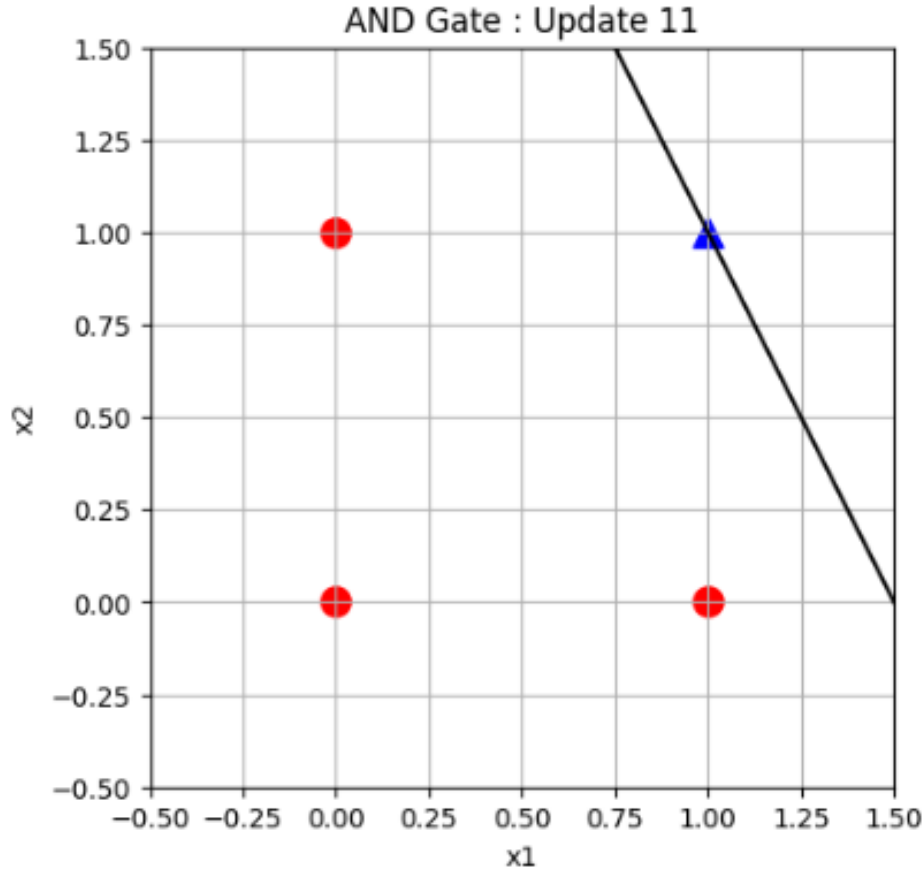
The NOT gate contains only one input, so the perceptron learns it very quickly. After just a small number of updates, the model finds the correct decision boundary and classifies both possible inputs correctly. The final weights are **[-1, 0]** with a bias of **0**, showing that the perceptron has converged successfully.

4. AND Gate – Update 1



In the first update, the decision boundary is still far from its final position, so several input combinations are classified incorrectly. At this point, the perceptron has only started adjusting its weights based on the training data and requires additional updates to improve its predictions.

5. AND Gate – Final Update



By the final update, the perceptron has learned the AND gate correctly. The decision boundary separates the single positive input (1,1) from the remaining three negative inputs without any errors. The model converges with final weights [2, 1] and a bias of -3, confirming that the AND gate is also linearly separable and can be learned using the perceptron algorithm.

Overall Observation

The implementation of different logic gates showed how the perceptron updates its weights and bias to learn a decision boundary. For the OR, NOT, and AND gates, the model gradually improved its predictions after each update and eventually classified all input combinations correctly. Since these gates are linearly separable, the perceptron was able to converge without difficulty. On the other side, the XOR gate could not be learned successfully because no single straight line can separate its output classes. This highlights one of the main limitations of a Single Layer Perceptron and explains why multilayer neural networks are needed for solving non-linear classification problems.

10. Conclusion

In this experiment, a single layer perceptron was implemented from scratch to classify the Banknote authentication dataset. Before training the model, the dataset was analyzed, normalized, and divided into the training and testing sets. The trained model achieved an accuracy of **98.18%**, along with high precision, recall, and F1-score, showing that it was able to classify the banknotes effectively.

The training process showed that the number of misclassified samples reduced as learning progressed, while the weights and bias gradually became stable after several epochs. Different learning

rates were also tested, and all of them produced comparable results because the dataset is nearly linearly separable. The implementation of the OR, NOT, AND, and XOR logic gates further improved the understanding of how the Perceptron learns through weight updates and where its limitations arise. Overall, this experiment provided practical knowledge of the Perceptron learning algorithm, decision boundaries, activation functions, model evaluation, and the importance of preprocessing in building an effective machine learning model.

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.